(Experiment 1-2]

[1, 2, 3, 4, 5]

[3, 5, 7, 9]

[8, 2, 6]

[0, 8]

[8]

So, the Num[i] = (num[i] + num[i+1]) % 10

we have to continue until only one element remains.

& then return the element.

- reduce the array step by step.
- At each step

  replace each element with the sum of adjacent % 10.
- continue until one element left.

Algo :- n = size of nums.

```
while(n>1):
    for i=0 to n-2:
        num[i] = (num[i] + nums[i+1]) % 10
    reduce size n = n-1

return nums[0]
```

Time complexity = n^2

Space = 1

optimal :- Based on Pascal's triangle

result = (result + (comb * num[i]) % 10

comb = Comb * (n-1-i) / i+1;

}

~~return res~~

code :-

```
int n = num.size();
int res = 0;
int comb = 1
for (int i = 0; i<n; i++) {
    res = (res + comb * num[i]) % 10;
    comb = comb * (n-1-i)/i+1;

}
return res;

}
```

time complexity = O(n)

Space complexity = O(1)